

4 RANK 04 > CPP (00 - 04) (DOING) Copy 

CPP01

Main topics

Let's go over the main topics that are present on the first page of the subject :

Memory allocation, pointers to members, references, switch statement

Memory allocation

Remember that C++ basically is an extension of C, yeah so memory allocation also exists in C++. It's mainly not done in the same way as in C.

In C we have `malloc` and `free`, in C++ we have `new` and `delete`. Let's take an example comparing C and C++ since you already know C memory management pretty well.

C Code

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  int main(void)
5  {
6      int *ptr; // declare the pointer
7
8      ptr = malloc(sizeof(int)); // allocate the memory
9      *ptr = 45; // assign a value
10     printf("ptr value: %d\n", *ptr); // print the value
11     free(ptr); // deallocate the memory
12     return (0);
13 }
```

C++ Code

```
1  #include <iostream>
2
3  int main(void)
4  {
5      int *ptr; // declare the pointer
6
7      ptr = new int; // allocate memory
8      *ptr = 45 // assign a value
9      std::cout << "ptr value: " << *ptr << std::endl; // print
the value
10     delete ptr; // deallocate memory
11     return (0);
12 }
```

In this case, both codes are exactly the same, now let's look at how you would do that for a dynamically allocated array.

C Code

```
1  #include <stdlib.h>
2
3  int main(void)
4  {
5      int *ptr;
6
7      ptr = malloc (10 * sizeof(int));
8      free(ptr);
9      return (0);
10 }
```

C++ Code

```
1  #include <iostream>
2
3  int main(void)
4  {
5      int *ptr;
6
7      ptr = new int[10];
8      delete [] ptr;
9      return (0);
10 }
```

Pointers to members

First of all, what are pointers to members ?

Remember that in C, we have pointers to variable and pointers to functions. These also exists in C++ but there's also 2 new pointer types in C++ to introduce the class concept into pointers.

I'll put down here the code example from the intra video on pointers to members.

```

1  #include <iostream>
2  #include <Sample.hpp>
3
4  int main()
5  {
6      Sample instance; // instance of Sample class on the stack
7      Sample *instancep = &instance; // pointer to the instance
      of Sample
8
9      int Sample::*p = NULL; // pointer to member attribute of
      Sample class
10     // adding the Sample:: specifies that it will point to a
      member attribute of the Sample class
11     void (Sample::*f)(void) const; // pointer to member
      function of Sample class
12
13     p = &Sample::foo; // Setting Sample::*p to be the address
      of the foo member attribute
14
15     std::cout << "Value of member foo: " << instance.foo <<
      std::endl;
16     instance.*p = 21; // this line sets the value of the
      member attribute pointed to by *p in the instance `instance`
      to 21
17
18     std::cout << "Value of member foo: " << instance.foo <<
      std::endl;
19     instancep->*p = 42; // this line sets the value of the
      member attribute pointed to by *p in the instance pointed to
      by the `instancep` pointer
20     std::cout << "Value of member foo: " << instance.foo <<
      std::endl;
21
22     f = &Sample::bar;
23
24     (instance.*f)(); // calling the member function Bar in
      the instance `instance` of the Sample class
25     (instancep->*f)(); // same thing as above, but this time
      with a pointer to the instance
26 }

```

.* operator

Refers to line 16 in the above example

We set `Sample::*p` to point to a member `foo` in the `Sample` class. The thing is, we could have multiple instances of the `Sample` class, so we have to specify on which instance we want to use the pointer by using the following syntax: `instance.*pointerToMemberAttribute`.

This is because the `foo` member attribute can exist in multiple class instances.

- >* operator

Refers to line 19 in the above example

Here we are not using the `Sample` instance directly anymore, we have to use the `->*` operator instead of the `.*` operator. It works exactly the same way but it adds a level of dereference in between.

This syntax: `instancePtr->*pointerToMemberAttribute` where `instancePtr` points to an `instance` of the `Sample` class, does the same thing as the `.*` operator used on instance variable directly.

Call member function from pointer to member function

Refers to line 24 and 25 in the above example

For pointers to member functions, it works in the same way as pointer to member attributes.

We first set to which member function we want the pointer to point, and when we want to call the function, we have to specify on which instance of the class we'll call the function.

In the same way as pointer to member attributes, we have the two operators, `.*` and `->*`.

The `.*` operator is used when we use the instance variable directly, and the `->*` is used when we have a pointer to an instance variable.

References

References are a new concept, that does not exist in `C`, it's close as what the pointers do in `C`.

The more accurate definition of a reference is the following:

A reference is a pointer that is constant and is always dereferenced, it can never be `void\`

Let's take the first example from the intra video

c++ Refs Uncommented

```
1  #include <iostream>
2
3  int main(void)
4  {
5      int numberOfBalls = 42;
6
7      int *ballsPtr = &numberOfBalls;
8      int &ballsRef = numberOfBalls;
9
10     std::cout << numberOfBalls << " " << *ballsPtr << " " <<
        ballsRef << std::endl;
11
12     *ballsPtr = 21;
13     std::cout << numberOfBalls << std::endl;
14     ballsRef = 84;
15     std::cout << numberOfBalls << std::endl;
16
17     return (0);
18 }
```

With just the definition given above, can you guess the output of the above code ?

≥ Code result

Now, we'll take the same example, but this time, I'll comment it so that we can understand what is being done in this code.

c++ Refs commented

```
1  #include <iostream>
2
3  int main(void)
4  {
5      int numberOfBalls = 42; // assigning 42 to the
        numberOfBalls variable
6
7      int *ballsPtr = &numberOfBalls; // assigning the address
        of the numberOfBalls variable to the ballsPtr pointer
8      int &ballsRef = numberOfBalls; // assigning ballsRef to
        be a reference to the numberOfBalls variable
9      // note that for reference, we assign it directly another
        variable and not the adress of another variable
10     // Now that the reference is set, we cannot change what
        it is referencing.
11     // This means that I could not do this later on: ballsRef
        = otherIntValue;
12
13     std::cout << numberOfBalls << " " << *ballsPtr << " " <<
        ballsRef << std::endl;
14     // The above line first prints the value of the
        numberOfBalls variable
15     // it then prints the value pointed to by ballsPtr by
        dereferencing the pointer using the * operator
16     // it then prints the value of ballsRef which is a
        reference to the numberOfBalls variable
17
18     *ballsPtr = 21;
19     // dereference the pointer to modify the value pointed to
        by ballsPtr
20     std::cout << numberOfBalls << std::endl;
21     // should print 21
22     ballsRef = 84;
23     // modify the value referenced by ballsRef
24     std::cout << numberOfBalls << std::endl;
25     // should print 84
26
27     return (0);
28 }
```

Things to note about references in C++

- A reference is somehow like a dereferenced pointer

- Once it's defined it will always be referencing the same value
- You cannot declare a reference without assigning it a value directly
- References are constant, you can't change what it references after the declaration
- A reference cannot be `void`, unlike a pointer that can be void

Some things of the above list in the form of code, as it can be better visualize by some:

```
1  #include <iostream>
2
3  int main(void)
4  {
5      int intValue = 0;
6      int secondValue = 10;
7
8      int &valRef = intValue; // Will always reference intValue
9      int &valRef2; // this will not work as references have to
    be directly assigned a variable to reference
10     valRef = secondValue; // this will not work as valRef was
    already set to reference intValue
11     int &valRef3 = NULL; // this will not work, first because
    references cannot be void, but also because we will not be
    able to set it to reference something else later since they
    are constant
12 }
```

Switch statement

There's already something about the `switch` statement that you can read [here](#).

Previous
CPP00

Next
CPP02

Last updated 1 year ago

Was this helpful?

